

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного образовательного
учреждения высшего образования
**«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В.ПЛЕХАНОВА»**

Техникум Пермского института (филиала)

Практическая работа
по дисциплине «Технология разработки программного
обеспечения»

по теме: «Система управления складом»

Руководитель:

/ Берестов Д.Б./ _____ -
«_____» _____-2026г.

Исполнитель:

/ Ощепков Д.С./ _____ -
«_____» _____-2026г.

Пермь, 2026

Оглавление

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»	1
ВВЕДЕНИЕ	3
РАЗРАБОТКА БИБЛИОТЕКИ WAREHOUSEDATA	4
1.1. Модели данных.....	4
1.1.1. Базовые справочники (Category, Manufacturer, Supplier).....	4
1.1.2. Организации и Склады (Organization, Warehouse).....	4
1.1.3. Товарная номенклатура (Product).....	5
1.1.4. Документооборот (Invoice, InvoiceItem).....	5
1.2. Контекст данных и инициализация.....	6
1.3. Сервисный слой.....	7
1.3.1. Сервис OrganizationService.....	7
1.3.2. Сервис ServiceWhs.....	7
1.3.3. Сервис ServiceInvoice.....	8
РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА	10
2.1. Глобальные стили и точка входа.....	10
2.2. Основные окна и навигация.....	10
2.3. Модальные окна и справочники.....	10
2.4. Вспомогательные классы.....	11
РАЗРАБОТКА МОДУЛЬНЫХ ТЕСТОВ	12
3.1. Тестирование сервиса накладных.....	12
3.2. Тестирование сервисов организаций и складов.....	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	14

ВВЕДЕНИЕ

В рамках данной практической работы было разработано настольное приложение «Warehouse» — система управления складским хозяйством. Инструментарий разработки включает среду Visual Studio, язык программирования C# и фреймворк WPF (.NET Framework).

Главной целью проекта является создание интуитивно понятного интерфейса для учета справочных данных (организаций, складов, номенклатуры) и документооборота (накладных). Архитектура приложения базируется на иерархической навигации: выбор организации →\rightarrow→ выбор склада →\rightarrow→ работа с товарами и накладными.

Для обеспечения масштабируемости и тестируемости применен принцип разделения ответственности. Бизнес-логика и модели данных вынесены в изолированную библиотеку классов WarehouseData. Хранение данных реализовано в оперативной памяти с использованием коллекций. Пользовательский интерфейс реализован в WPF-проекте Warehouse и взаимодействует с логическим слоем через внедрение зависимостей. Корректность работы бизнес-логики подтверждена модульными тестами (MSTest).

РАЗРАБОТКА БИБЛИОТЕКИ WAREHOUSEDATA

Библиотека WarehouseData представляет собой фундаментальный слой приложения, инкапсулирующий все модели предметной области, контекст хранения данных и сервисы бизнес-логики. Архитектурное решение вынести эти компоненты в отдельный проект (Class Library) обусловлено принципом разделения ответственности (Separation of Concerns): пользовательский интерфейс WPF-проекта Warehouse не должен зависеть от деталей хранения данных, а бизнес-логика должна оставаться тестируемой независимо от графического представления.

Пространство имен библиотеки структурировано по папкам: Models, Context, Services. Несмотря на то, что в текущей реализации данные хранятся в оперативной памяти, модели помечены атрибутами Entity Framework ([Table], [Key], [Column], [ForeignKey], [DatabaseGenerated]), что обеспечивает готовность архитектуры к бесшовной миграции на реляционную СУБД (например, MS SQL Server) без изменения кода бизнес-логики.

1.1. Модели данных

Все сущности расположены в пространстве имен WarehouseData.Models и представляют собой POCO-классы (Plain Old CLR Objects), совместимые с ORM-фреймворками.

1.1.1. Базовые справочники (Category, Manufacturer, Supplier)

Три справочных класса имеют унифицированную структуру, что обеспечивает единообразие при работе с ними через универсальные окна (DictionaryWindow).

Класс Category соответствует таблице categories. Содержит два свойства: Id (первичный ключ с автоинкрементом через [DatabaseGenerated(DatabaseGeneratedOption.Identity)]) и Name (название категории, помечено null! для обозначения обязательности). Атрибут [Table("categories")] задает явное соответствие таблице БД. Пустой конструктор необходим для работы EF при материализации объектов.

Класс Manufacturer (таблица manufacturers) и **класс Supplier** (таблица suppliers) реализованы по идентичной схеме. Они хранят названия производителей (например, «LG», «Bosch», «Xiaomi») и поставщиков (например, «ООО ТехноИмпорт», «ООО ОфисЛайн») соответственно.

1.1.2. Организации и Склады (Organization, Warehouse)

Эти классы образуют иерархию верхнего уровня: организация владеет одним или несколькими складами.

Класс Organization (таблица orgs) использует статическое поле OrgCounter типа BigInteger для генерации уникальных идентификаторов. Свойство OrgId является первичным ключом, а в конструкторе дополнительно выполняется ручное присваивание OrgId = ++OrgCounter для работы в in-memory режиме. Ключевым навигационным свойством является warehouses типа ObservableCollection<Warehouse>, что позволяет UI автоматически отслеживать добавление и удаление складов. Конструктор принимает строковое имя организации.

Класс Warehouse (таблица warehouses) аналогично использует статический счетчик WarehousesCounter. Содержит свойства WhName (название), WhAddress (почтовый адрес) и OrgId (внешний ключ к организации). Атрибут [ForeignKey("OrgId")] на навигационном свойстве Organization явно задает связь «многие-к-одному». Коллекция products типа List<Product> хранит номенклатуру конкретного склада.

1.1.3. Товарная номенклатура (Product)

Класс Product (таблица products) является наиболее сложной моделью. Обязательные строковые поля: Article (артикул), Name (наименование), Unit (единица измерения). Финансовые и учетные показатели: Price (тип decimal для точного хранения денежных значений), DiscountPercent (процент скидки), StockQuantity (текущий остаток).

Внешние ключи CategoryId, ManufacturerId, SupplierId связывают товар со справочниками, а навигационные свойства Category, Manufacturer, Supplier обеспечивают удобный доступ к связанным объектам. Дополнительные поля Description (описание) и PhotoPath (путь к изображению) являются опциональными (nullable). Поле SupplierName инициализируется пустой строкой для быстрого отображения в UI без дополнительных запросов.

1.1.4. Документооборот (Invoice, InvoiceItem)

Для учета движения товаров реализована система накладных.

Перечисление InvoiceType определяет два направления движения: Incoming (приходная) и Outgoing (расходная).

Класс Invoice использует статический счетчик _counter для генерации уникальных идентификаторов. Свойства: Id, Number (внешний номер документа), Type (тип накладной), Date (дата создания, по умолчанию DateTime.Now), IsConfirmed (флаг проведения, по умолчанию false), Items (список позиций типа List<InvoiceItem>). Флаг IsConfirmed играет ключевую

роль в защите данных: подтвержденную накладную нельзя удалить или повторно провести.

Класс InvoiceItem представляет одну строку документа. Свойство `Product` имеет тип `Product?` (nullable reference type), что отражает возможность создания позиции до полной инициализации. Свойство `Quantity` хранит количество единиц товара.

1.2. Контекст данных и инициализация

Класс `ApplicationContext` (пространство имен `WarehouseData.Context`) выступает в роли репозитория in-memory данных. Он содержит пять публичных полей типа `ObservableCollection<T>`:

- `orgs` — организации;
- `whs` — склады;
- `categories` — категории товаров;
- `manufacturers` — производители;
- `suppliers` — поставщики.

Использование `ObservableCollection` вместо стандартных `List<T>` является принципиальным решением: коллекции автоматически генерируют события `CollectionChanged`, на которые подписывается WPF-интерфейс. Это обеспечивает реактивное обновление таблиц и списков без необходимости вручную вызывать методы перерисовки.

В конструкторе контекста вызывается метод `OrgsFill()`, который заполняет хранилище тестовыми данными для демонстрации функциональности:

- 4 категории (Электроника, Канцелярские товары, Бытовая техника, Продукты питания);
- 5 производителей (LG, Xiaomi, ErichKrause, Bosch, Nestlé);
- 3 поставщика (ООО «ТехноИмпорт», ООО «ОфисЛайн», ООО «ПродТорг»);
- 2 организации («Северстрой, ООО» и «Гарант-Торг, ООО»);
- 4 склада (по 2 на каждую организацию с почтовыми адресами).

Каждый склад наполняется специфичной номенклатурой: центральный склад «Северстроя» содержит электронику (телевизор LG, смартфон и планшет Xiaomi), северный — бытовую технику Bosch, основной склад «Гарант-Торга» — канцелярию ErichKrause, розничный — продукты Nestlé. Все товары получают полные атрибуты: артикулы (например, EL-001), цены, остатки, скидки и ссылки на связанные сущности.

1.3. Сервисный слой

Сервисы (пространство имен WarehouseData.Services) реализуют паттерн Service Layer, инкапсулируя бизнес-правила и предоставляя UI единый API для работы с данными. Зависимости внедряются через конструкторы (Dependency Injection), что обеспечивает слабую связанность компонентов.

1.3.1. Сервис OrganizationService

Отвечает за управление организациями. В конструкторе принимает ApplicationContext через внедрение зависимости и сохраняет его в приватном поле `_context`.

Ключевые методы:

- **GetContext()** — возвращает ссылку на контекст данных для прямого доступа к коллекциям из UI;
- **AddOrg(Organization org)** — добавляет организацию с валидацией (проверка на null и пустое название). Возвращает bool для индикации успеха;
- **EditOrg(Organization org)** — ищет организацию по OrgId через FirstOrDefault и обновляет название;
- **DelOrg(Organization org)** — удаляет организацию из коллекции с предварительной проверкой на null.

Все методы возвращают булевы значения, что позволяет UI отображать сообщения об ошибках без использования исключений для управления потоком.

1.3.2. Сервис ServiceWhs

Управляет складами и товарами. В конструкторе принимает ApplicationContext и текущую Organization. Хранит три поля: `_context`, `_org` (текущая организация) и `_currentWarehouse` (выбранный склад).

Методы управления складами:

- **SetCurrentWarehouse(Warehouse wh)** — устанавливает активный склад;
- **GetCurrentWarehouseProducts()** — возвращает товары текущего склада (или пустой список, если склад не выбран);
- **GetWarehouses()** — возвращает все склады организации через ToList();
- **AddWh(Warehouse wh), EditWh(Warehouse wh), DelWh(Warehouse wh)** — CRUD-операции над складами с валидацией.

Методы управления товарами:

- **AddProduct(Warehouse wh, Product product)** — добавляет товар на конкретный склад;
- **EditProduct(Warehouse wh, Product product)** — ищет товар по артикулу через FirstOrDefault и обновляет все свойства (название, цену, остаток, скидку, связанные сущности);
- **RemoveProduct(Warehouse wh, Product product)** — удаляет товар со склада.

1.3.3. Сервис ServiceInvoice

Центральный сервис документооборота, реализующий ключевую бизнес-логику проведения накладных. В конструкторе принимает ServiceWhs через внедрение зависимости, что обеспечивает доступ к текущему складу и его товарам.

Ключевые методы:

- **GetInvoices()** — возвращает накладные как IEnumerable<Invoice>, скрывая возможность прямого изменения коллекции;
- **Add(Invoice invoice)** — добавляет накладную с проверкой на null;
- **Delete(Invoice invoice)** — удаляет только неподтвержденные накладные (защита от потери данных по проведенным документам);
- **Confirm(Invoice invoice)** — главный метод проведения документа.

Алгоритм метода Confirm:

1. Проверяет, что накладная не null и не была проведена ранее (IsConfirmed == false);
2. Получает текущий список товаров через `_serviceWhs.GetCurrentWarehouseProducts()`;
3. В цикле по позициям накладной (Items) находит соответствующий товар на складе по артикулу;
4. Для приходной накладной (InvoiceType.Incoming) увеличивает StockQuantity товара на значение Quantity из позиции;
5. Для расходной накладной (InvoiceType.Outgoing) уменьшает StockQuantity;
6. Если товар не найден на складе или позиция содержит null-товар — позиция пропускается (защита от сбоев);
7. После обработки всех позиций устанавливает IsConfirmed = true.

Такой подход гарантирует атомарность проведения: либо все позиции обработаны и документ проведен, либо (при наличии ошибок валидации)

операция прерывается. Повторный вызов Confirm для уже проведенной накладной возвращает false, что предотвращает двойное списание/оприходование.

РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Интерфейс реализован в WPF-проекте Warehouse с использованием глобальных стилей, команд и конвертеров.

2.1. Глобальные стили и точка входа

В App.xaml подключен словарь ресурсов DictionaryBasic.xaml, задающий единую цветовую палитру (тёмно-синее меню, светло-серый фон) и стили для всех элементов управления. В App.xaml.cs инициализируется ApplicationContext, создается OrganizationService и открывается стартовое окно OrgsView.

2.2. Основные окна и навигация

- **OrgsView:** Стартовое окно выбора организации. Содержит ListBox с контекстным меню для CRUD-операций. При подтверждении выбора открывается MainWindow (см.рисунок 1).

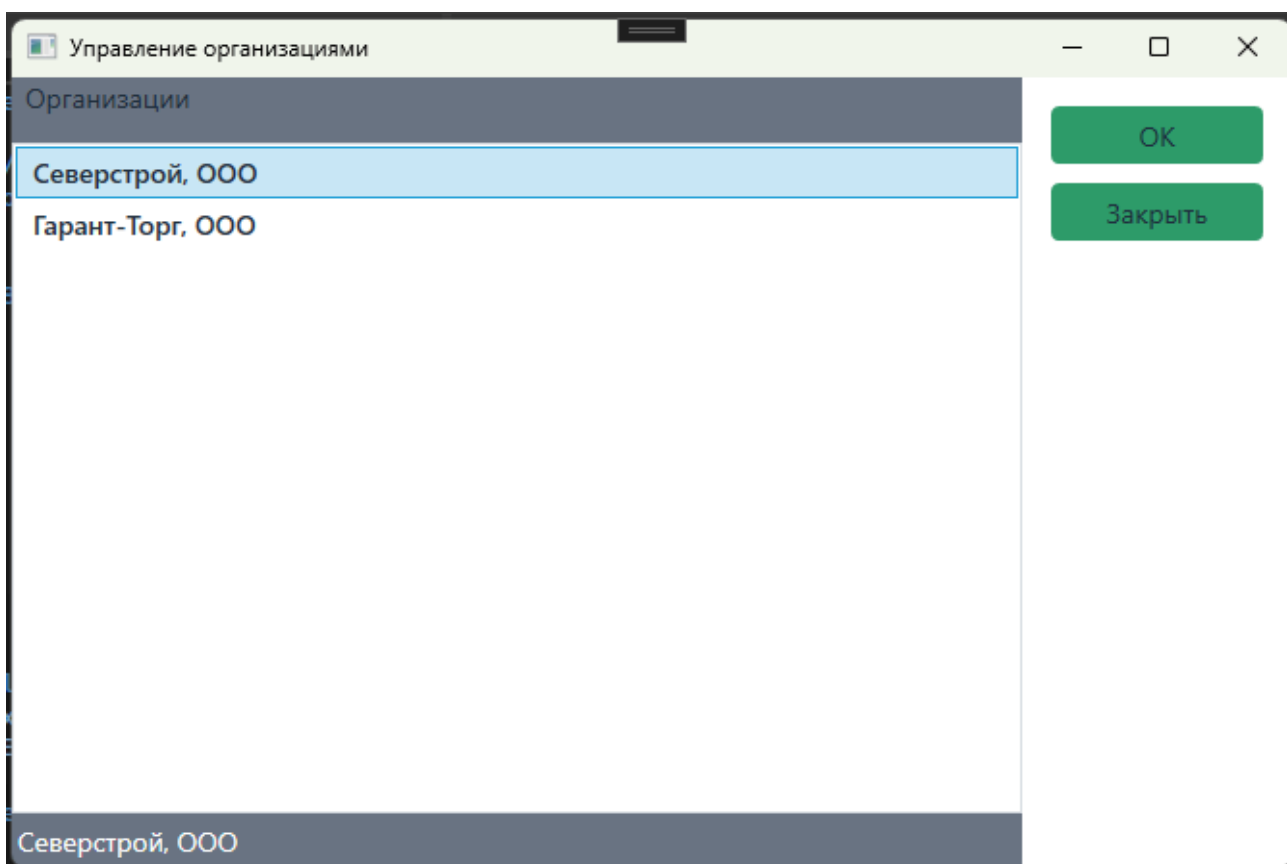


Рисунок 1 - Стартовое окно выбора организации

- **MainWindow:** Главный контроллер приложения. Использует DockPanel и Frame. В Frame по умолчанию загружается UserControlWarehouses. Меню делегирует команды элементу управления.
- **UserControlWarehouses:** Рабочее пространство, отображающее список складов и таблицу товаров выбранного склада (см.рисунок 2). Инкапсулирует методы обработки команд для товаров и накладных.

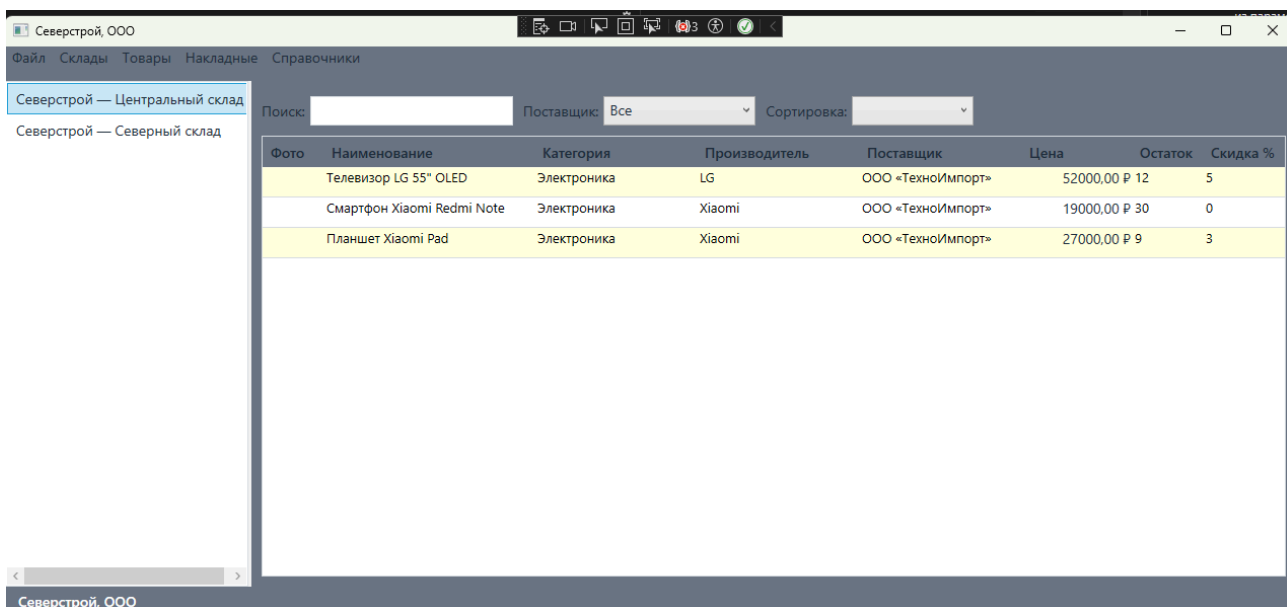


Рисунок 2- Главный контроллер приложения

2.3. Модальные окна и справочники

Для ввода и редактирования данных разработана система модальных окон:

- **OrgModWindow / WhModWindow:** Окна ввода данных организаций и складов (см.рисунок 3). Поддерживают режимы добавления, редактирования и удаления (с требованием точного ввода имени для подтверждения удаления).

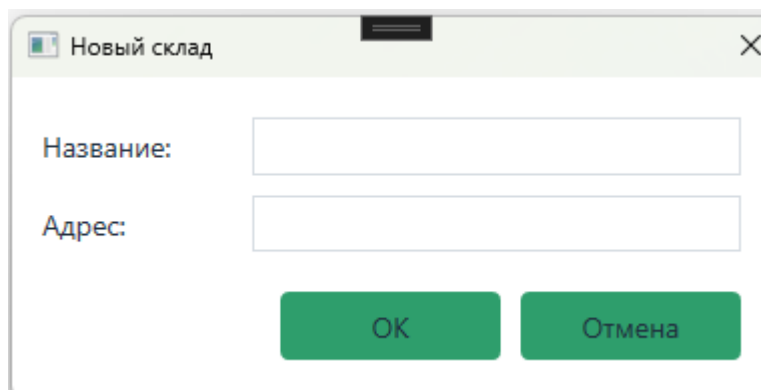


Рисунок 3- Окно ввода данных организаций и складов

- **ProductModWindow:** Форма ввода товара с валидацией (цена ≥ 0 , скидка 0-100%) и выпадающими списками (ComboBox) для выбора связанных сущностей.
- **DictionaryWindow и DictionaryItemWindow:** Универсальные окна для управления справочниками (категории, поставщики, производители) с использованием dynamic для работы с любыми коллекциями.
- **Окна накладных:** InvoiceWindow (список накладных и их состав), InvoiceCreateWindow (выбор типа и даты), InvoiceItemWindow (добавление позиции) (см.рисунок 4).

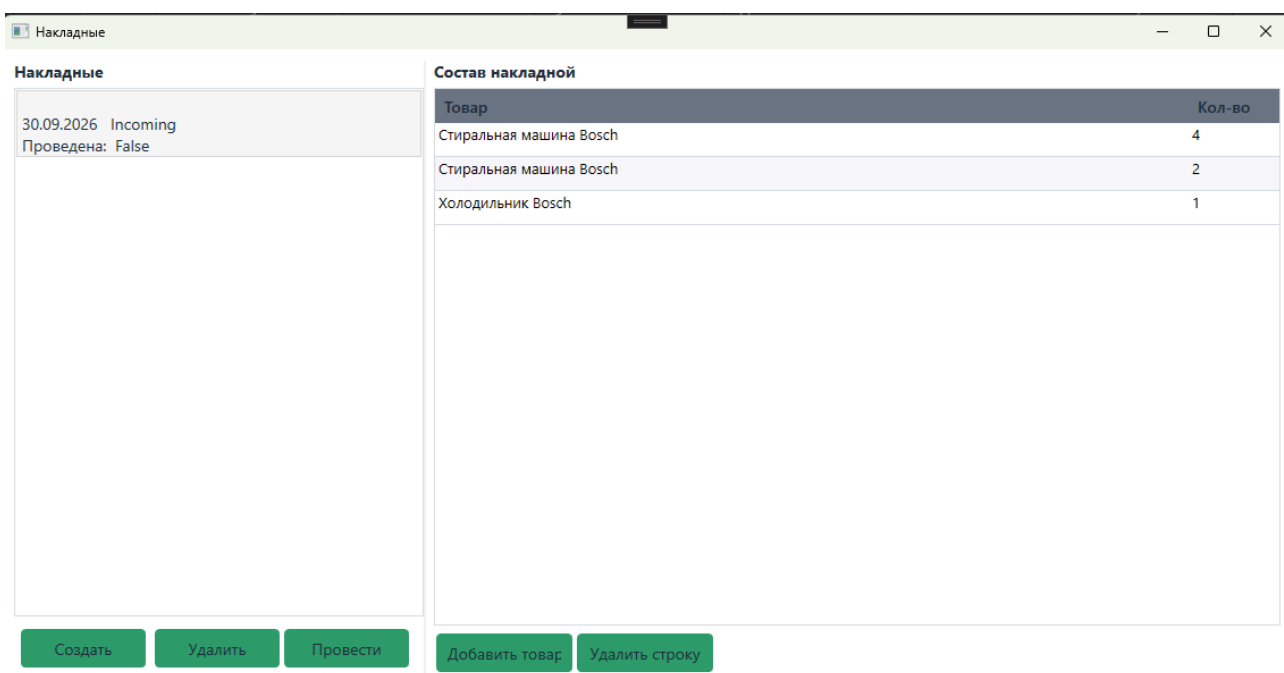


Рисунок 4- Окно накладных

2.4. Вспомогательные классы

- **Конвертеры (Converters.cs):** PriceConverter форматирует цену (2 знака после запятой, символ ₽). DiscountBackgroundConverter подсвечивает товары со скидкой светло-желтым цветом.
- **Команды (WindowCommands.cs):** Статический класс, централизующий 19 RoutedCommand. Это позволило отвязать UI-разметку от бизнес-логики, обеспечив слабую связанность компонентов.

РАЗРАБОТКА МОДУЛЬНЫХ ТЕСТОВ

Для верификации бизнес-логики создан проект WarehouseData.Tests с использованием фреймворка MSTest. Тесты покрывают позитивные сценарии, валидацию и краевые случаи (см.рисунок 5).

3.1. Тестирование сервиса накладных

Написано 11 тестов. Проверяется:

- Добавление и удаление накладных (включая запрет на удаление подтвержденных и обработку null).
- Логика проведения (Confirm): увеличение остатков для Incoming, уменьшение для Outgoing.
- Защита от повторного проведения.
- Обработка ошибок: пропуск позиций с null-товаром или товаром, отсутствующим на складе.

3.2. Тестирование сервисов организаций и складов

- **OrganizationServiceTests (11 тестов):** Проверка CRUD-операций, валидации пустых имен, обработки null и ссылочной идентичности контекста.
- **ServiceWhsTests (15 тестов):** Проверка управления складами, установки текущего склада, добавления/редактирования/удаления товаров, а также обработка некорректных аргументов.

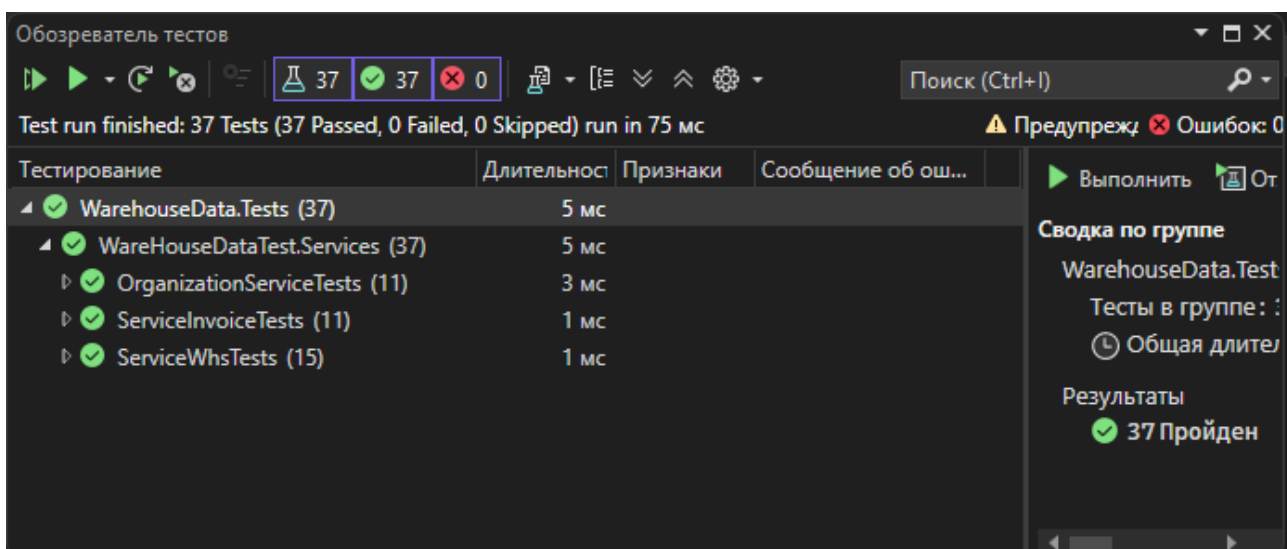


Рисунок 5- Прохождение тестов

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы было успешно разработано настольное приложение «Warehouse» для автоматизации складского учета. Ключевым достижением является применение архитектурного паттерна разделения ответственности. Вынос бизнес-логики в библиотеку WarehouseData позволил сделать пользовательский интерфейс легковесным, а саму логику — независимой и легко тестируемой. Использование ObservableCollection и внедрения зависимостей обеспечило надежную связь между слоями.

Реализованный функционал покрывает полный цикл складского учета: от ведения справочников и управления иерархией «Организация – Склад – Товар» до оформления и проведения приходных/расходных накладных с автоматическим пересчетом остатков.

Написание модульных тестов (MSTest) позволило выявить и устранить логические ошибки на этапе разработки, гарантировав стабильность работы системы. Приложение обладает интуитивно понятным интерфейсом, единой стилистикой и готово к дальнейшему масштабированию, например, к подключению реляционной базы данных благодаря заложенным атрибутам Entity Framework.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Microsoft Learn. Документация по языку программирования C# [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 26.09.2026).

Microsoft Learn. Модульное тестирование кода C# с помощью MSTest [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest> (дата обращения: 26.09.2026).

Metanit.com. Полное руководство по языку программирования C# и платформе .NET [Электронный ресурс]. – URL: <https://metanit.com/sharp/tutorial/> (дата обращения: 26.09.2026).

Metanit.com. Руководство по WPF [Электронный ресурс]. – URL: <https://metanit.com/sharp/wpf/> (дата обращения: 26.09.2026).